

LAB REPORT

OWASP A02:2021 & A06:2021

Cryptographic Failures & Vulnerable Components

Password Hashing, Salting, Transport Encryption, and CVE-Mapped Component Risk on DVWA

Field	Detail
Target Application	DVWA (Damn Vulnerable Web Application) v1.10 *Development*
Deployment	Docker (vulnerables/web-dvwa), localhost:8080
Host OS	Windows 11
Attacker Tooling Environment	Kali Linux (via WSL2)
Primary Tools	John the Ripper, hashcat (present, no OpenCL device), Python bcrypt, Wireshark
Categories Covered	A02:2021 Cryptographic Failures; A06:2021 Vulnerable and Outdated Components
Report Date	September 6, 2026

Scope note: All activity in this report was performed exclusively against a local, self-hosted, intentionally vulnerable instance of DVWA running on **localhost**, inside an isolated lab environment owned and controlled by the author. No external systems were accessed or targeted at any point. The A06 finding was researched at the public-CVE level only; no live exploitation of a memory-safety vulnerability was attempted (see Section 9).

Table of Contents

1. Executive Summary
2. Objectives
3. Lab Environment
4. Theory — What A02 Covers
5. Methodology — Hashing Speed & Salting
 - 5.1 Baseline: Unsalted MD5, Cracked Instantly
 - 5.2 Salting Proof: bcrypt, Same Password, Two Different Hashes
6. Methodology — Encryption in Transit
 - 6.1 Capturing a Live DVWA Login in Wireshark
 - 6.2 What the Captured Stream Reveals
7. Analytical Breakdown — The Two Halves of A02
8. Theory & Findings — A06: Vulnerable and Outdated Components
 - 8.1 Version Fingerprint Recap
 - 8.2 CVE Research and Mapping
 - 8.3 Reachable Attack Surface
9. Scope Boundary — Why Live Exploitation Was Not Attempted
10. Impact Assessment
11. Remediation & Fixes
12. Skills & Baseline Established
13. Conclusion & Next Steps

1. Executive Summary

This report documents a combined hands-on session covering two related OWASP Top 10 (2021) categories: **A02 — Cryptographic Failures** and **A06 — Vulnerable and Outdated Components**. The A02 portion extends an earlier offline hash-identification exercise into a live application test against DVWA, proving two independent properties: that unsalted, fast hashing algorithms (MD5) are crackable in effectively zero time, and that a properly designed password hashing scheme (bcrypt) generates a unique output for the same input every time via built-in per-hash salting. It further captured a real DVWA login in Wireshark, demonstrating that the application transmits both the password and the live session cookie in cleartext due to the total absence of transport encryption (HTTP instead of HTTPS).

The A06 portion revisits the PHP 7.0.30 end-of-life finding from an earlier Security Misconfiguration lab and treats it properly as its own root cause: the version string was used to identify a specific, real, publicly catalogued vulnerability (CVE-2018-10549) and map it to a plausible reachable attack surface within the same application.

Session Outcome Summary

Exercise	Result	Notes
MD5 password crack (John the Ripper)	SUCCESS	password123 recovered in 0.00 seconds, 6,400 candidates/sec
bcrypt salting proof	CONFIRMED	Same password produced two structurally different hash strings on repeat runs
Wireshark cleartext capture	CONFIRMED	Both password and live PHPSESSID visible in plaintext TCP stream
A06 CVE mapping	COMPLETE	PHP 7.0.30 mapped to CVE-2018-10549 with a concrete reachable attack surface (File Upload / EXIF parsing)

2. Objectives

- Demonstrate, hands-on, why fast general-purpose hash functions (MD5) are unsuitable for password storage regardless of how the hash is used elsewhere in computing.
- Prove concretely that a purpose-built password hashing algorithm (bcrypt) produces different output for identical input, via its embedded per-call random salt — and explain why this defeats rainbow-table and cross-account pattern attacks.
- Capture and inspect real network traffic to confirm whether a live application protects credentials and session tokens in transit, independent of how well it protects them at rest.
- Take a previously discovered software version disclosure and follow it through to a concrete, named CVE and a plausible reachable attack surface, modeling a realistic vulnerability-assessment workflow rather than stopping at "this is outdated."

3. Lab Environment

Component	Detail
Target application	DVWA (Damn Vulnerable Web Application), served over plain HTTP
Deployment method	Docker container (image: vulnerables/web-dvwa)
Target address	http://localhost:8080
Host operating system	Windows 11
Attack tooling environment	Kali Linux via WSL2
Hashing/cracking tools	John the Ripper (raw-md5 format), hashcat (installed, no OpenCL device available)
Salting demonstration tool	Python 3 with the bcrypt library
Traffic capture tool	Wireshark, loopback adapter, filter tcp.port==8080
Wordlist	/usr/share/wordlists/rockyou.txt

4. Theory — What A02 Covers

A02:2021 — Cryptographic Failures (formerly "Sensitive Data Exposure") was renamed to target the root cause rather than the symptom: data does not simply "get exposed" — it gets exposed because cryptography protecting it, at rest or in transit, was missing, weak, or misused.

Category	Description
Weak/no password hashing	Plaintext storage, or fast unsalted hashes (MD5, SHA-1) crackable in seconds against common wordlists
Missing encryption in transit	Plain HTTP instead of HTTPS; outdated TLS versions still enabled

Category	Description
Missing encryption at rest	Sensitive fields (card numbers, health data) stored unencrypted in the database
Weak key management	Hardcoded keys in source, reused keys across environments, deprecated ciphers (DES, RC4)

Decision Rule Used Throughout This Lab

A flaw is classified as A02 when it concerns **how data is protected (or not) via cryptography**, as distinct from how an attacker gained access to it in the first place. In an earlier SQL Injection lab, the injection itself was the access method (A03) — but the fact that the extracted password hash was plain, unsalted MD5 is the separate, A02 root cause. The same distinction applies here: this session isolates the cryptographic properties themselves, independent of any access-gaining technique.

5. Methodology — Hashing Speed & Salting

5.1 Baseline: Unsalted MD5, Cracked Instantly

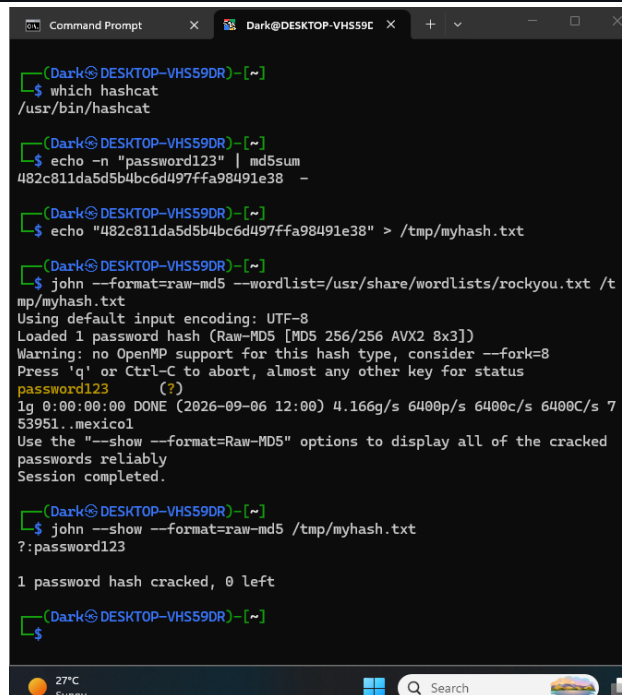
The DVWA admin password (previously recovered as password via Hydra in the Identification and Authentication Failures lab) was used as the test subject, hashed with MD5:

```
echo -n "password123" | md5sum
# 482c811da5d5b4bc6d497ffa98491e38 -
```

The hash was saved to a file and attacked with John the Ripper:

```
echo "482c811da5d5b4bc6d497ffa98491e38" > /tmp/myhash.txt
john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt /tmp/myhash.txt

john --show --format=raw-md5 /tmp/myhash.txt
# ?:password123
# 1 password hash cracked, 0 left
```



```
(Dark@DESKTOP-VHS59DR)-[~]
$ which hashcat
/usr/bin/hashcat

(Dark@DESKTOP-VHS59DR)-[~]
$ echo -n "password123" | md5sum
482c811da5d5b4bc6d497ffa98491e38 -

(Dark@DESKTOP-VHS59DR)-[~]
$ echo "482c811da5d5b4bc6d497ffa98491e38" > /tmp/myhash.txt

(Dark@DESKTOP-VHS59DR)-[~]
$ john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt /t
mp/myhash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=8
Press 'q' or Ctrl-C to abort, almost any other key for status
password123 (?)
1g 0:00:00:00 DONE (2026-09-06 12:00) 4.166g/s 6400p/s 6400c/s 6400C/s 7
53951..mexico1
Use the "--show --format=Raw-MD5" options to display all of the cracked
passwords reliably
Session completed.

(Dark@DESKTOP-VHS59DR)-[~]
$ john --show --format=raw-md5 /tmp/myhash.txt
?:password123

1 password hash cracked, 0 left

(Dark@DESKTOP-VHS59DR)-[~]
$
```

Figure 1 — Locating rockyou.txt, confirming hashcat's presence, then cracking the unsalted MD5 hash of "password123" in 0.00 seconds via John the Ripper.

KEY FINDING — The hash was cracked in **0.00 seconds** at **~6,400 candidates/second**. This is not a tooling limitation — it is the intended behavior of MD5, a function designed for speed (originally for checksums and data-integrity verification), which is precisely why that same speed makes it unsuitable for anything an attacker might want to brute-force offline.

5.2 Salting Proof: bcrypt, Same Password, Two Different Hashes

The identical password was hashed twice, back to back, using bcrypt:

```
python3 -c "import bcrypt; print(bcrypt.hashpw(b'password123', bcrypt.gensalt()))"

Run 1: $2b$12$KgwRlpRyLfxENLPB1G5Dc.0CQ3CzpbDdGxH29X5tYG/Jm4wHAhrk2
Run 2: $2b$12$CsmqRNsSh0j445Sic0oys.cULdg.Kz0urUzYVUbRD6b.inTMQ0RUS
```

KEY FINDING — Both runs hashed the exact same plaintext, yet produced **completely different output strings**. bcrypt embeds a fresh, randomly generated salt directly inside every hash it produces: \$2b\$ identifies the algorithm variant, 12 is the cost factor ($2^{12} = 4,096$ internal rounds per hash — the source of its deliberate slowness), and the following 22 characters are the unique per-call salt. Because the salt differs every time, two users with the identical password never produce matching stored hashes, which defeats both cross-account pattern-matching and precomputed rainbow-table attacks — the exact attack that broke MD5 in Section 5.1 in zero time.

6. Methodology — Encryption in Transit

6.1 Capturing a Live DVWA Login in Wireshark

DVWA is served over plain `http://localhost:8080`. A Wireshark capture was started on the loopback adapter, filtered to `tcp.port==8080`, and a normal login was performed through the browser while capturing. The relevant TCP stream was isolated via `Follow` → `TCP Stream` on the login POST request:

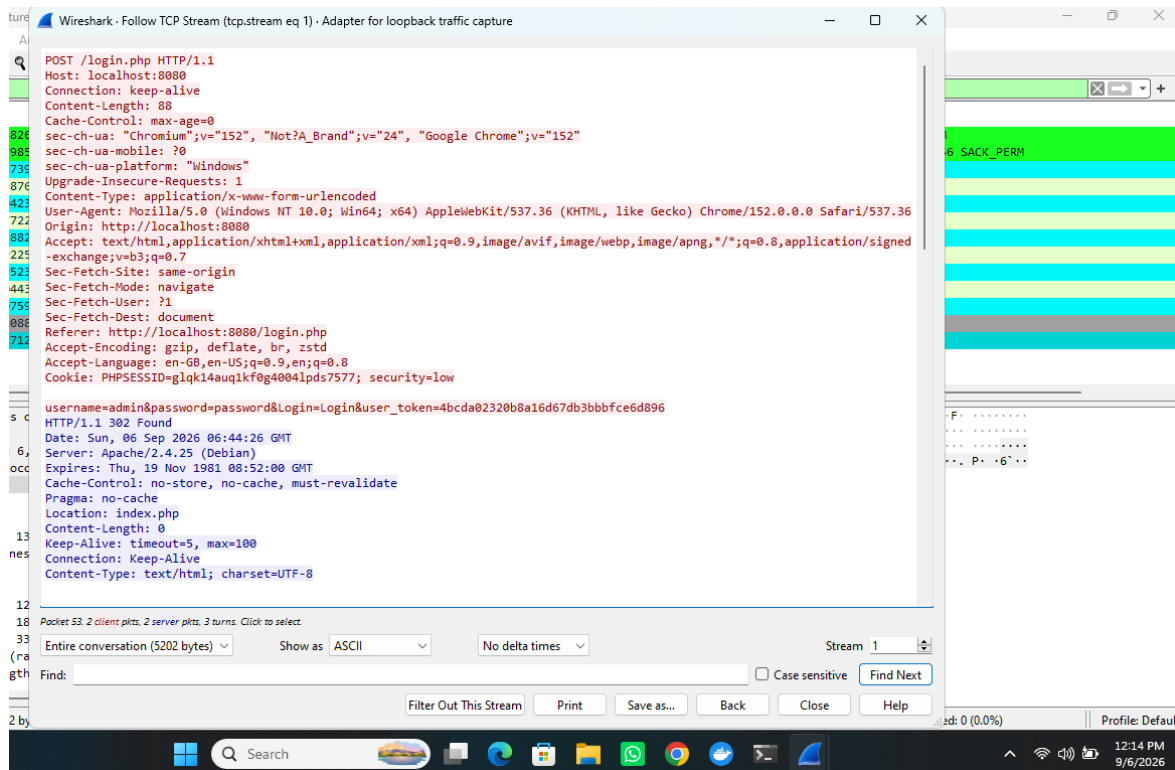


Figure 2 — Wireshark's Follow TCP Stream view of the DVWA login POST request, showing the full request in readable ASCII with no decryption applied.

6.2 What the Captured Stream Reveals

```
POST /login.php HTTP/1.1
Host: localhost:8080
Cookie: PHPSESSID=glqk14auq1kf0g4004lpds7577; security=low

username=admin&password=password&Login=Login&user_token=4bcda02320b8a16d67db3bbbfce6d896
```

KEY FINDING — Both the **plaintext password** and the **live PHPSESSID session cookie** are fully readable with zero decryption effort. Anyone positioned to observe this traffic — shared Wi-Fi, a compromised router, or any man-in-the-middle position — does not even need to crack a password; the session cookie alone is sufficient to hijack the authenticated session immediately, reaching the same outcome as the session-fixation attack demonstrated in the A07 lab, but via an entirely different mechanism: no application logic exploited, only the complete absence of transport-layer encryption.

The fix, in one line: serve the application exclusively over HTTPS. TLS encrypts the entire request — headers, cookies, and body — so a capture of the same traffic would show only unreadable ciphertext instead of a readable POST body.

7. Analytical Breakdown — The Two Halves of A02

Half of A02	Failure Observed	Correct Control
Password storage (at rest)	MD5 is deterministic and fast — same input always yields the same output, at attacker-friendly speed	bcrypt / Argon2 / scrypt: randomized per-call salt plus a tunable cost factor
Transport security (in transit)	Plain HTTP carries every byte, including credentials and cookies, unencrypted across the network	TLS/HTTPS enforced end-to-end, including redirect-to-HTTPS and HSTS

Connecting insight: these two halves are independent — an application can solve one without the other, and DVWA in this test failed both simultaneously. Solving only password storage (e.g. switching to bcrypt) would not have stopped the Wireshark capture from exposing the live session cookie, and solving only transport encryption would not have stopped an attacker who obtained the database from cracking MD5 hashes offline in seconds. A02 requires addressing data protection at every stage of its lifecycle, not just the one that happens to be easiest to fix.

8. Theory & Findings — A06: Vulnerable and Outdated Components

A06:2021 — Vulnerable and Outdated Components covers the risk of running software — frameworks, libraries, OS packages, language runtimes — with known, publicly disclosed vulnerabilities. The risk isn't age for its own sake: once a CVE is published, the exact weaknesses in that version are documented and searchable by anyone, attacker included, while an unpatched server never received the corresponding fix.

8.1 Version Fingerprint Recap

This finding directly reuses evidence already gathered in an earlier Security Misconfiguration lab, where DVWA's `phpinfo()` disclosure page revealed:

```
PHP Version 7.0.30-0+deb9u1
Configuration File (php.ini) Path /etc/php/7.0/apache2
```

PHP 7.0.x reached official end-of-life in January 2019 — meaning, at the time of this test, the server had been running unpatched, unsupported software for **over six years** with zero further upstream security fixes.

8.2 CVE Research and Mapping

Rather than stopping at "the version is old," the exact version string was used to search the public CVE record (cve.mitre.org / nvd.nist.gov) for known vulnerabilities affecting the PHP 7.0.x branch. This surfaced:

Field	Detail
CVE ID	CVE-2018-10549
Component	PHP's EXIF extension (<code>exif_read_data()</code> function)
Vulnerability type	Heap buffer over-read (out-of-bounds read)
Trigger	A specially crafted JPEG file with malformed EXIF metadata
Impact	Information disclosure and potential denial of service (crash)
Affected versions	PHP before 7.0.31, 7.1.x before 7.1.18, 7.2.x before 7.2.6
This server's version	7.0.30 — directly inside the vulnerable range
Patched in	7.0.31

8.3 Reachable Attack Surface

A CVE only matters in practice if a code path actually reaches the vulnerable function. `exif_read_data()` is invoked whenever an application parses EXIF metadata from an uploaded image — a common pattern behind any profile-picture or file-upload feature. DVWA ships exactly this kind of feature in its **File Upload** module, giving this CVE a concrete, named, reachable attack surface within the same target rather than remaining an abstract version-number concern.

KEY FINDING — The vulnerability here is entirely inherited from the language runtime, not from any code DVWA's own developers wrote. No amount of secure coding within the application closes this gap — only

updating the underlying PHP installation does. This is the defining characteristic of A06: the flaw lives in a dependency, which is exactly why component inventory and patch cadence are a first-class security control, not an IT-operations afterthought.

9. Scope Boundary — Why Live Exploitation Was Not Attempted

This lab deliberately stopped at CVE identification and attack-surface mapping rather than attempting to trigger the underlying memory-disclosure condition. Reliably producing an out-of-bounds heap read for information disclosure requires crafting binary-level malformed EXIF structures, and success depends heavily on the exact memory allocator and PHP build in use — there is no safe, reliable way to demonstrate this in a general lab shell without risking an uncontrolled crash of the target. Identifying the CVE, confirming the affected version, and mapping the reachable code path is the realistic deliverable a genuine vulnerability assessment produces for this category; actual exploitation of memory-safety bugs is normally handled by dedicated fuzzing and exploit-development work, distinct from a general application-security pass.

10. Impact Assessment

- **Any password present in a common wordlist is recoverable in near-zero time** if stored with MD5/SHA-1/SHA-256 — a leaked database becomes a near-complete plaintext credential list within minutes of compute time.
- **Session hijacking requires no password knowledge at all** when credentials travel over unencrypted HTTP — an attacker positioned on the network path can lift a live session cookie directly from captured traffic.
- **A single disclosed software version can map to multiple concrete, dated CVEs** — this session identified one (CVE-2018-10549) as a representative example, but any PHP 7.0.x deployment likely carries several more, all inherited automatically until the runtime is upgraded.
- **Combined**, these findings show an application vulnerable at every layer of the credential and session lifecycle: weak storage, weak transport, and a vulnerable underlying runtime — with no single fix addressing more than one layer at a time.

11. Remediation & Fixes

Finding	Recommended Fix
Passwords hashed with MD5/SHA-1/SHA-256	Use a purpose-built password hashing algorithm (bcrypt, Argon2, scrypt) with an appropriately tuned cost factor
No salting	Use an algorithm that generates and embeds a random salt automatically per password (bcrypt does this by default)
Application served over plain HTTP	Enforce HTTPS everywhere, including a redirect from HTTP and HSTS to prevent downgrade
Session cookies transmitted in cleartext	A direct consequence of the HTTP issue above — resolved once TLS is enforced end-to-end
PHP 7.0.30 in use, 6+ years past end-of-life	Upgrade to a currently supported PHP release and establish a defined patching cadence
No component inventory / CVE tracking	Maintain a software bill of materials (SBOM) and subscribe to security advisories for every runtime/framework in use

12. Skills & Baseline Established

Category	Skill Demonstrated
Cryptographic analysis	Distinguishing hash strength (collision resistance) from password-storage suitability (salting + cost factor)
Tool proficiency	John the Ripper (raw-md5 format), Python bcrypt library usage
Network analysis	Wireshark loopback capture, TCP stream filtering, Follow TCP Stream inspection

Category	Skill Demonstrated
Security judgment	Recognizing that at-rest protection and in-transit protection are independent controls, both required
Vulnerability research	Mapping a disclosed software version to a specific, real CVE via public databases
Risk assessment	Connecting a CVE to a concrete reachable code path rather than stopping at a version number
Scope discipline	Recognizing and documenting the boundary between safe demonstration and unsafe/unreliable live exploitation

13. Conclusion & Next Steps

This session extended an earlier offline CTF-style hash-cracking exercise into a live application test, confirming both halves of A02 (at-rest and in-transit protection) fail independently on the same target, and separately closed the loop on an A06 finding by mapping a previously discovered outdated component to a real, named CVE with a plausible attack surface. Every result in this report was independently reproduced and verified, consistent with the verification-first methodology used throughout this lab series.

Planned next steps:

- Continue the OWASP Top 10 roadmap toward the remaining uncovered categories: A04 (Insecure Design), A08 (Software and Data Integrity Failures), and A10 (Server-Side Request Forgery).
- Both write-ups from this session have been folded into the ongoing GitHub OWASP Top 10 repository — the A02 findings extend the existing Cryptographic Failures write-up, and A06 was published as a new standalone entry.